

OUTLIER DETECTION & DATABASE UPDATE SCRIPT

End-to-End Technical & Business Logic Explanation

Python + Pandas + NumPy + PostgreSQL

Primary purpose	Calculate transaction rate, detect outliers, classify records, and update public.transactions.
Run scope	City-level, with optional location-level filtering.
Special geography logic	Dubai and Abu Dhabi use city-wide location grouping; Mumbai has special exception rules.
Final database fields	rate, is_outlier, outlier_type

1. Purpose of the Script

This script performs a complete outlier-processing cycle for a selected city and, optionally, for a selected location such as Andheri. It reads transaction data from PostgreSQL, calculates a rate per square foot, builds statistically comparable groups, derives robust group limits, applies business-specific rules, classifies every transaction, and writes the result back to the same transaction row.

```
PostgreSQL -> Fetch transactions -> Calculate rate -> Group records -> P1/P99 trimming -> Median -> Limits ->
Classify -> Update PostgreSQL
```

2. Configuration and Run Scope

```
CITY_ID = 8
LOCATION_NAME = "Andheri"
```

The script is configured before execution. CITY_ID determines the city to process. LOCATION_NAME optionally restricts the run to one exact location inside that city. When LOCATION_NAME is None, the whole city is processed.

Setting	Meaning
DB_PARAMS	PostgreSQL host, port, database, user and password.
CITY_ID	The city whose transactions will be processed.
LOCATION_NAME	Optional location filter. Example: Andheri, Baner.
GROUP_COLS	Grouping keys used for percentile and median calculations.

3. Entry Point and Main Control Flow

```
if __name__ == "__main__":
    main(city_id=CITY_ID, location_name=LOCATION_NAME)
```

Execution starts in main(). The function controls the full pipeline in this order:

1. Connect to PostgreSQL.
2. Fetch the required transactions.
3. Compute rates, statistics and outlier flags.
4. Bulk update the transactions table.
5. Print a processing summary.
6. Close the database connection even if an error occurs.

4. Database Connection

```
conn = psycopg2.connect(**DB_PARAMS)
```

psycpg2 creates a live connection between Python and PostgreSQL. The same connection is passed into fetch and update functions and is closed in the finally block.

5. Fetching Transactions

fetch_data() reads only the columns required for outlier processing. The city name is obtained through a join with dim_city.

```
SELECT
    t.transaction_id, c.city_name, t.location_name,
    t.transaction_category, t.property_type,
    t.agreement_price, t.net_carpet_area_sq_m
FROM public.transactions t
JOIN public.dim_city c ON c.city_id = t.city_id
WHERE t.city_id = %s
```

Column	Why it is required
transaction_id	Unique key used to update the same transaction later.
city_name	Used for city-specific rules such as Mumbai, Dubai and Abu Dhabi.
location_name	Used to form comparable market groups.
transaction_category	Separates Sale, Lease, etc.
property_type	Separates apartment, commercial, plot, etc.
agreement_price	Used in rate calculation and Mumbai low-price rule.
net_carpet_area_sq_m	Used to calculate rate per sq.ft.

6. Location Filtering and Dubai / Abu Dhabi Rule

For ordinary cities, a non-null LOCATION_NAME adds a case-insensitive, trim-safe location condition. For Dubai and Abu Dhabi, the script intentionally ignores a supplied location filter because the later grouping logic treats all locations as one city-wide market bucket.

```
AND lower(trim(t.location_name)) = lower(trim(%s))
```

This means Andheri, ANDHERI, and values with leading/trailing spaces are treated as equivalent when fetching.

7. DataFrame Creation

pd.read_sql() converts the PostgreSQL result into a Pandas DataFrame. All subsequent statistical work is performed in memory before the final update is sent back to the database.

8. Normalization and Validity Checks

The script normalizes city and transaction-category text by trimming spaces and converting values to lowercase. It then identifies valid rows.

```
is_valid = (agreement_price > 0) AND (net_carpet_area_sq_m > 0)
```

Agreement Price	Area	Valid for Statistics?
> 0	> 0	Yes
0	> 0	No
> 0	0	No
NULL	> 0	No
> 0	NULL	No

9. Rate Calculation

For valid rows, the rate is calculated as price divided by carpet area converted from square metres to square feet.

```
calc_rate = agreement_price / (net_carpet_area_sq_m * 10.764)
```

Example: Agreement price = Rs. 1,00,00,000 and area = 100 sq.m. Area in sq.ft = 100 x 10.764 = 1,076.4 sq.ft. Rate = 1,00,00,000 / 1,076.4 = approximately Rs. 9,290/sq.ft. Invalid rows receive NaN for calc_rate.

10. Statistical Group Construction

The script creates comparable groups using four keys: city, location, transaction category and property type. This prevents fundamentally different transaction populations from influencing one another.

```
GROUP_COLS = ["k_city", "k_loc", "k_cat", "k_ptype"]
```

Example Group	Interpretation
Mumbai Andheri Sale Apartment	Apartment sale transactions in Andheri.
Mumbai Andheri Sale Commercial	Commercial sale transactions in Andheri.
Mumbai Andheri Lease Apartment	Apartment lease transactions in Andheri.

Missing grouping values are replaced by ~NULL~ so that they remain groupable rather than silently disappearing from group-by operations.

11. Dubai / Abu Dhabi Group Location Collapse

For Dubai and Abu Dhabi, every location is replaced by __ALL_LOCATIONS__ for grouping. Therefore Business Bay, Dubai Marina, Downtown and other locations all contribute to the same statistical location bucket for that city.

12. P1 and P99 Calculation

Only valid rows participate in the statistical calculation. Within each group, the script calculates the 1st percentile (P1) and 99th percentile (P99) of calc_rate.

```
valid_df.groupby(GROUP_COLS)["calc_rate"].quantile([0.01, 0.99])
```

P1 and P99 are not the final outlier limits. They are used only to temporarily trim extreme values before calculating the median, so that very abnormal records do not distort the group centre.

13. Trimming and Median Calculation

```
trimmed = rows where p1 <= calc_rate <= p99
```

The median of the trimmed rows is then calculated for each group. Median is preferred over mean because it is more resistant to unusually large or small transaction values.

14. Final Lower and Upper Limits

The final statistical outlier band is created from the trimmed median:

```
lower_limit = median_rate / 4  
upper_limit = median_rate * 4
```

Example: if the trimmed median is Rs. 12,000/sq.ft, the lower limit is Rs. 3,000/sq.ft and the upper limit is Rs. 48,000/sq.ft.

Rate	Classification against example limits
Rs. 2,500/sq.ft	Lower Outlier
Rs. 11,000/sq.ft	Within range
Rs. 60,000/sq.ft	Upper Outlier

15. Base Outlier Flags

Flag	Condition
is_lower	Valid transaction and calc_rate < lower_limit.
is_upper	Valid transaction and calc_rate > upper_limit.
is_rate_null	calc_rate is NaN / could not be calculated.
is_mumbai_low_price	Mumbai + Sale + agreement_price < Rs. 1,00,000.
is_mumbai_zero_exception	Mumbai + null/zero rate + zero/null area or price.

16. Mumbai Low-Price Rule

A Mumbai Sale transaction with agreement price below Rs. 1,00,000 is specifically flagged by business rule, independently of the statistical rate limits.

```
city = Mumbai AND category = Sale AND agreement_price < 100000
```

17. Mumbai Zero-Area / Zero-Rate Exception

Normally, a row with no calculable rate would become an outlier. However, the script creates a specific Mumbai exception: if the record has a null/zero rate because price or area is zero/null, it is forced to NOT be an outlier and receives a descriptive classification.

```
outlier_type = "Mumbai Zero Area & Rate - Not Outlier"  
is_outlier = False
```

18. Final Outlier Decision

The final Boolean outlier flag follows this decision rule:

```
IF Mumbai zero exception:  
    is_outlier = False  
ELSE:  
    is_outlier = is_lower OR is_upper OR is_rate_null OR is_mumbai_low_price
```

19. Outlier Type Assignment

A readable outlier_type is assigned in a fixed priority order. The order matters because the first matching condition wins.

Priority	Condition	Assigned Type
1	Mumbai zero exception	Mumbai Zero Area & Rate - Not Outlier
2	Not valid	Invalid/Excluded
3	Mumbai low-price rule	Mumbai Low Price Outlier (Sale < 1L)
4	Lower statistical outlier	Lower Outlier
5	Upper statistical outlier	Upper Outlier
6	No rule matched	Normal

Important: because Invalid/Excluded is checked before the Mumbai low-price label, a low-price Mumbai record that is also invalid can receive Invalid/Excluded rather than the low-price type.

20. Final Result DataFrame

Only four fields are retained for the database update: transaction_id, rate, is_outlier and outlier_type. Pandas NaN rate values are converted to Python None so PostgreSQL receives SQL NULL.

transaction_id	rate	is_outlier	outlier_type
1001	12500	False	Normal

1002	2100	True	Lower Outlier
1003	72000	True	Upper Outlier
1004	NULL	False	Mumbai Zero Area & Rate - Not Outlier

21. Bulk Database Update

`update_db()` converts each result row into a tuple and uses `psycopg2.extras.execute_values()` to perform an efficient bulk update instead of sending one UPDATE statement per transaction.

```
UPDATE public.transactions AS t
SET rate = data.rate,
    is_outlier = data.is_outlier,
    outlier_type = data.outlier_type
FROM (VALUES %s) AS data(transaction_id, rate, is_outlier, outlier_type)
WHERE t.transaction_id = data.transaction_id
```

The WHERE condition ensures that the calculated output is written back to the same existing transaction using `transaction_id`. No new transaction is inserted.

22. Batch Size and Commit

`execute_values()` uses a page size of 5,000, which keeps large updates manageable. After the update completes, `conn.commit()` permanently saves the changes in PostgreSQL.

23. Final Console Summary

After processing, the script counts True values in `is_outlier` and prints the scope, number of updated rows and number of detected outliers. Example:

```
city_id=8, location='Andheri': updated 3836 rows (14 flagged as outliers).
```

24. Complete End-to-End Flow

1. Start script
2. Read CITY_ID and optional LOCATION_NAME
3. Connect to PostgreSQL
4. Get city name
5. Apply or ignore location filter
6. Fetch transactions
7. Create DataFrame

8. Normalize city/category text
9. Validate price and area
10. Calculate rate per sq.ft
11. Create comparison groups
12. Calculate group P1/P99
13. Trim rates outside P1-P99 for median calculation
14. Calculate trimmed median
15. Calculate lower limit = median / 4
16. Calculate upper limit = median x 4
17. Apply lower/upper/null-rate rules
18. Apply Mumbai low-price rule
19. Apply Mumbai zero-area/zero-rate exception
20. Assign is_outlier and outlier_type
21. Build update result
22. Bulk update public.transactions
23. Commit
24. Print summary
25. Close connection

25. Worked Example

Assume a Mumbai-Andheri Sale-Apartment transaction has agreement price Rs. 1,20,00,000 and carpet area 100 sq.m.

Area in sq.ft = $100 \times 10.764 = 1,076.4$

Rate = $1,20,00,000 / 1,076.4 = \text{approx. Rs. } 11,148/\text{sq.ft}$

Assume the trimmed median for the same statistical group is Rs. 12,000/sq.ft. Then lower limit = Rs. 3,000/sq.ft and upper limit = Rs. 48,000/sq.ft. Since Rs. 11,148 is inside the final range and no Mumbai special rule applies, the transaction is classified as Normal with is_outlier = False.

26. Most Important Concept

Range	Purpose
P1 to P99	Used only to clean the dataset before calculating the median.
Median/4 to Median x 4	Used to classify lower and upper statistical outliers.

Therefore, a transaction outside the P1-P99 band is not automatically an outlier. The actual lower/upper outlier decision is based on the median-derived limits, together with the special business rules.

27. Output Fields Updated in PostgreSQL

Field	Stored value
rate	Calculated rate per sq.ft, or NULL when unavailable.
is_outlier	Boolean True/False final classification.
outlier_type	Readable classification or reason.

28. Summary

The script is a complete read-calculate-classify-write pipeline. It calculates comparable market statistics at group level, protects the median from extreme tails using P1/P99 trimming, applies wide median-based limits for final statistical outlier detection, incorporates custom city-specific rules, and updates the original transaction records efficiently in PostgreSQL.